

Динамически подключаемые библиотеки DLL

DLL

DLL (Dynamic Link Library) - это библиотека, которая содержит код и данные, которые могут быть использованы несколькими программами одновременно. Они используются в операционных системах Windows и позволяют разделить программный код на отдельные, многократно используемые компоненты.

DLL

До появления DLL библиотек, для использования кода из других программ, нужно было включать этот код в каждую программу отдельно, что приводило к дублированию кода и занимало много места в памяти.

Пример

Библиотека `kernel32.dll` содержит важные функции ядра ОС, которые используются практически всеми приложениями для взаимодействия с системой.

Пример

Примеры функций, предоставляемых библиотекой:

- `CreateProcess()`: создание нового процесса
- `ExitProcess()`: завершение текущего процесса
- `ReadFile()`, `WriteFile()`: чтение и запись данных в файлы
- `Sleep()`: приостановка выполнения потока на время

```
using System.Diagnostics;
try
{
    Process.Start("notepad.exe");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

Консоль отладки Microsoft Visual

C:\Users\304.PC-KLASS.000\
работу с кодом 0 (0x0).
Нажмите любую клавишу, что

Безымянный – Блокнот

Файл Правка Формат Вид Справка

Пример

Здесь мы используем класс `Process` из пространства имен `System.Diagnostics`, который инкапсулирует работу с процессами в .NET Framework. Метод `Start()` автоматически вызывает функцию `CreateProcess()` из `kernel32.dll`.

Пример

С помощью библиотеки DLL программу можно разделить на отдельные компоненты. Каждый модуль можно загрузить в основную программу во время выполнения, если он установлен. Так как модули являются отдельными, время загрузки программы ускоряется. Модуль загружается только тогда, когда запрашивается эта функциональность.

Пример

Например мы создаем программу для обработки заказов клиентов. Разделим на модули:

- Core.dll: общие утилиты и базовые классы
- DataAccess.dll: работа с базой данных
- BusinessLogic.dll: логика обработки заказов
- UI.dll: пользовательский интерфейс

Преимущества использования DLL

- Модульность
- Повторное использование кода
- Упрощенное тестирование
- Улучшенная поддержка
- Использует меньше ресурсов

Недостатки использования DLL

- Зависимость от внешних библиотек
- Риск конфликта версий

Зависимости DLL

Если программа или библиотека DLL использует функцию DLL в другой библиотеке DLL, создается зависимость. Программа больше не является автономной, и при нарушении зависимости могут возникнуть проблемы.

Типы DLL библиотек

Библиотеки бывают разных видов в зависимости от того, каким образом они связаны с приложением и какие задачи выполняют.

1. Статически связываемые

Такие библиотеки интегрируются в приложение на этапе компиляции. При сборке проекта компилятор включает код из DLL непосредственно в исполняемый файл. Это обеспечивает высокую производительность, но размер исполняемого файла увеличивается и любые изменения в библиотеке требуют перекомпиляции приложения.

2. Динамически связываемые

Самый распространенный вид DLL. Библиотека загружается в память только тогда, когда она необходима приложению. Динамическое связывание происходит во время выполнения программы, что позволяет уменьшить размер исполняемых файлов и упростить обновление библиотек.

2. Динамически связываемые

Существует два способа динамического связывания:

- Раннее связывание: Загрузка DLL автоматически при старте программы
- Позднее связывание: библиотеки загружаться по мере необходимости.

3. Native DLL

Библиотеки написаны на нативных языках программирования, таких как C или C++ и предназначены для работы с ОС на низком уровне. Они обеспечивают прямой доступ к аппаратуре и системным ресурсам. Например, `kernel32.dll`

4. Resource-only DLL

Эти библиотеки содержат только ресурсы, такие как иконки, строки, изображения и другие данные, используемые приложением. Такие библиотеки не содержат исполняемого кода и служат исключительно для хранения ресурсов, которые могут быть локализованы или изменены без перекомпиляции основного приложения.

Другие ОС

Понятие динамической библиотеки (DLL) не является уникальным для Windows. DLL - это стандартный механизм для обеспечения переиспользования кода в различных операционных системах.

Другие ОС

В Unix-подобных операционных системах (например, в Linux и macOS) используется аналогичный механизм, называемый "разделяемая библиотека" (shared library). В Linux, shared libraries имеют расширение ".so" (от "shared object"), а в macOS - ".dylib" (от "dynamic library").

Альтернативы DLL

- Статические библиотеки
- Компонентные объекты
- Web-сервисы
- Модули или плагины

Создание DLL на C++

1. Создание проекта



Статическая библиотека

Сборка LIB-файла, который может быть упакован внутрь других исполняемых файлов Windows.

C++

Windows

Библиотека



Проект общего назначения

Настроить новый проект

Статическая библиотека

C++

Windows

Библиотека

Имя проекта

StaticLibrary

Расположение

C:\Users\NEO1\Documents\C++

...

Имя решения ⓘ

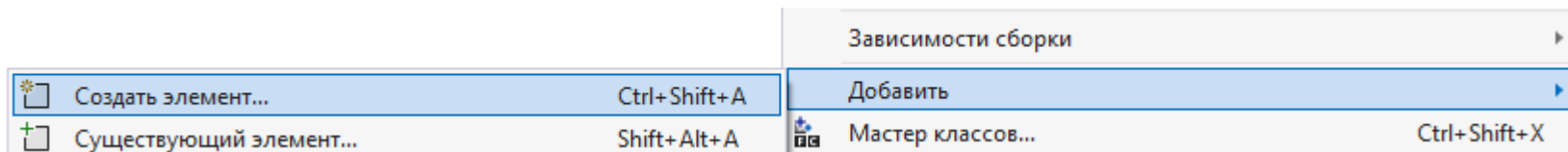
StaticLibrary

☐ Поместить решение и проект в одном каталоге

Проект будет создан в "C:\Users\NEO1\Documents\C++\StaticLibrary\StaticLibrary\"

2. Добавление кода

Добавить в код .crr файл, содержащий реализации функций.



Установленные

Visual C++

Код

Форматирование

Библиотека ATL

Данные

Ресурсы

Веб

Служебные программы

Таблицы свойств

Тест

HLSL

Графика

В сети

Сортировка: По умолчанию



Поиск (Ctrl+E)



	Файл C++ (.cpp)	Visual C++
	Файл заголовка (.h)	Visual C++
	Файл Midl (.idl)	Visual C++
	Файл определения модуля (.def)	Visual C++
	Класс C++	Visual C++
	Блок интерфейса модуля C++ (.ixx)	Visual C++

Тип: Visual C++

Создает файл, содержащий исходный код C++.

Имя:

StaticLibrary.cpp

Расположение:

C:\Users\NEO1\Documents\C++\StaticLibrary\StaticLibrary\

Обзор...

Показать компактное представление

Добавить

Отмена

```
1  // StaticLibrary.cpp : Определяет функции для статической библиотеки.
2  //
3
4  #include "pch.h"
5  #include "framework.h"
6
7  // TODO: Это пример библиотечной функции.
8  void fnStaticLibrary()
9  {
10 }
11 //наша функция
12 int sum(int a, int b) {
13     return a + b;
14 }
15
```

Установленные

Сортировка: По умолчанию



Поиск (Ctrl+E)



Visual C++

Код

Форматирование

Библиотека ATL

Данные

Ресурсы

Веб

Служебные программы

Таблицы свойств

Тест

HLSL

Графика

В сети



Файл C++ (.cpp)

Visual C++



Файл заголовка (.h)

Visual C++



Файл Midl (.idl)

Visual C++



Файл определения модуля (.def)

Visual C++



Класс C++

Visual C++



Блок интерфейса модуля C++ (.ixx)

Visual C++

Тип: Visual C++

Создает заголовочный файл C++

Имя:

StaticLibrary.h

Расположение:

C:\Users\NEO1\Documents\C++\StaticLibrary\StaticLibrary\

Обзор...

Показать компактное представление

Добавить

Отмена

StaticLibrary.h* × StaticLibrary.cpp*

StaticLibrary

(Глобал

```
1  #pragma once
2  //прототип функции
3  int sum(int a, int b);
```

3. Компилирование проекта

Нажать Ctrl+Shift+B или Сборка → Собрать решение
После успешной сборки с папке bin\Debug (bin\Release) появится файл ИмяБиблиотеки.lib. Это и есть LIB-библиотека.

Этот компьютер > Документы > C++ > StaticLibrary > x64 > Debug

Имя

Дата изменения

Тип

Размер

 StaticLibrary.idb

07.10.2025 21:34

VC++ Minimum R...

27 КБ

 StaticLibrary.lib

07.10.2025 21:34

Object File Library

9 КБ

 StaticLibrary.pdb

07.10.2025 21:34

Program Debug D...

76 КБ

 useLibrary.exe

07.10.2025 21:34

Приложение

66 КБ

 useLibrary.pdb

07.10.2025 21:34

Program Debug D...

1 188 КБ

4. Использование созданной DLL

В том же решении добавить новый проект, в котором будем использовать свою библиотеку.



Консольное приложение

Выполнить код в терминале Windows. По умолчанию выводится фраза "Hello World".

C++

Windows

Консоль

Конфигурация: Активная (Debug) v

Платформа: Активная (x64) v

Диспетчер конфигураций...

▲ Свойства конфигурации ^

Общие

Дополнительно

Отладка

Каталоги VC++

▷ C/C++

▲ Компоновщик

Общие

Ввод

Файл манифеста

Отладка

Система

Оптимизация

Внедренный IDL

Метаданные Wind

Дополнительно

Все параметры

Командная строка

▷ Инструмент манифес

▷ Генератор XML-докум

▷ Информация об исхо

▷ События сборки

Выходной файл \$(OutDir)\$(TargetName)\$(TargetExt)

Отображать ход выполнения Не задано

Версия

Включить инкрементную компоновку Да (/INCREMENTAL)

Файл базы данных инкрементной ком \$(IntDir)\$(TargetName).ilk

Отключить загрузочный баннер Да (/NOLOGO)

Пропустить библиотеку импорта Нет

Зарегистрировать вывод Нет

Перенаправление для пользователей Нет

Дополнительные каталоги библиотек C:\Users\NEO1\Documents\C++\StaticLibrary\x64\Debug;%

Компоновать библиотеки из зависимс Да

Использовать входные файлы зависи Нет

Состояние ссылки

Запретить привязку Dll

Обрабатывать предупреждения компс

Принудительный вывод файла

Создать оперативно исправляемый об

Задать атрибуты секции

Выходной файл

Параметр /OUT переопределяет стандартное имя и место программы, создаваемые компоновщиком.

OK

Отмена

Применить

Конфигурация: Активная (Debug) v

Платформа: Активная (x64) v

Диспетчер конфигураций...

Свойства конфигурации ^

Общие

Дополнительно

Отладка

Каталоги VC++

▷ C/C++

Компоновщик

Общие

Ввод

Файл манифеста

Отладка

Система

Оптимизация

Внедренный IDL

Метаданные Windows

Дополнительно

Все параметры

Командная строка

▷ Инструмент манифеста

▷ Генератор XML-документа

▷ Информация об исходных данных

▷ События сборки v

Дополнительные зависимости

"C:\Users\NEO1\Documents\C++\StaticLibrary\x64\Debug\StaticLibrary.lib"

Игнорировать все стандартные библиотеки

Игнорировать конкретные стандартные библиотеки

Файл определения модуля

Добавить модуль в сборку

Внедрить управляемый файл ресурсов

Принудительно включать ссылки на символы

Отложено загружаемые DLL

Ресурс, связываемый со сборкой

Дополнительные зависимости

Задаёт дополнительные элементы, которые нужно добавить в командную строку компоновки. (т.е. kernel32.lib)

OK

Отмена

Применить

Конфигурация: Активная (Debug) ▾

Платформа: Активная (x64) ▾

Диспетчер конфигураций...

Свойства конфигурации ^

Общие

Дополнительно

Отладка

Каталоги VC++

C/C++

Общие

Оптимизация

Препроцессор

Создание кода

Язык

Предварительно о

Выходные файлы

Информация об и

Внешние директив

Дополнительно

Все параметры

Командная строка

▷ Компоновщик

▷ Инструмент манифес

▷ Генератор XML-докум

▷ Информация об исхо

Дополнительные каталоги включаемых C:\Users\NEO1\Documents\C++\StaticLibrary\StaticLibrary; ^

Дополнительные каталоги #using

Дополнительные каталоги BMI

Дополнительные зависимости модуля

Дополнительные зависимости блока загл

Проверка источников на наличие зависи Нет

Преобразовать директивы include в дире Нет

Формат отладочной информации База данных программы для операции "Изменить и продол

Поддержка отладки только собственногос Да (/JMC)

Поддержка общезыковой среды выпол

Использовать расширение среды выпол

Отключить загрузочный баннер Да (/nologo)

Уровень предупреждений Уровень3 (/W3)

Обрабатывать предупреждения как оши(Нет (/WX-)

Версия предупреждений

Формат диагностики Информация о столбцах (/diagnostics:column)

Проверки SDL Да (/sdl)

Многопроцессорная компиляция

Включить санитайзер адресов Нет

Дополнительные каталоги включаемых файлов

Указывает один или несколько каталогов, добавляемых к пути поиска включаемых файлов; если необходимо указать несколько каталогов, разделите их точками с запятой (";"). (/I[путь])

OK

Отмена

Применить

```
[- #include <iostream>
```

```
  #include "StaticLibrary.h"
```

```
[- int main()
```

```
{
```

```
  std::cout << sum(a: 2, b: 4);
```

```
}
```